SignEase — Unified Sign Language Detection System

Complete Project Documentation

Author: Nachiket Shinde — KodeNeurons
Version: 2.0
Date: March 2026

Table of Contents

---

1. Project Overview

SignEase is a real-time, AI-powered sign language detection and translation system that bridges the communication gap between hearing-impaired individuals and the general public. It supports **two distinct sign languages** — **American Sign Language (ASL)** and **Indian Sign Language (ISL)** — within a single unified web application.

The system captures live webcam video, processes hand gestures using **Google MediaPipe** for hand landmark detection, classifies gestures using trained machine learning models, and builds readable sentences character by character. Users can then speak the constructed sentence aloud using the built-in Text-to-Speech engine or correct it using the **NVIDIA NIM API** (meta/llama model) for intelligent auto-correction.

Version 2.0 Update: Rebranded from "SignVerse" to "SignEase". Replaced Google Gemini AI with NVIDIA NIM API. Complete frontend redesign with premium glassmorphism UI, animated mesh background, gradient border system, and improved TTS reliability (subprocess-based engine for repeated button presses).

---

2. Problem Statement

According to the World Health Organization, over **466 million** people worldwide have disabling hearing loss. Sign language is the primary mode of communication for the deaf community, yet the vast majority of the hearing population cannot understand it. This creates a significant barrier in everyday interactions — from healthcare consultations to job interviews.

Existing solutions commonly:

- Support only a single sign language (typically ASL)
- Require specialized hardware (gloves, depth cameras)
- Lack real-time processing capability
- Do not provide sentence-level output or AI-based correction

There is a critical need for a **low-cost, real-time, multi-language** sign language detection system that requires only a standard webcam.

---

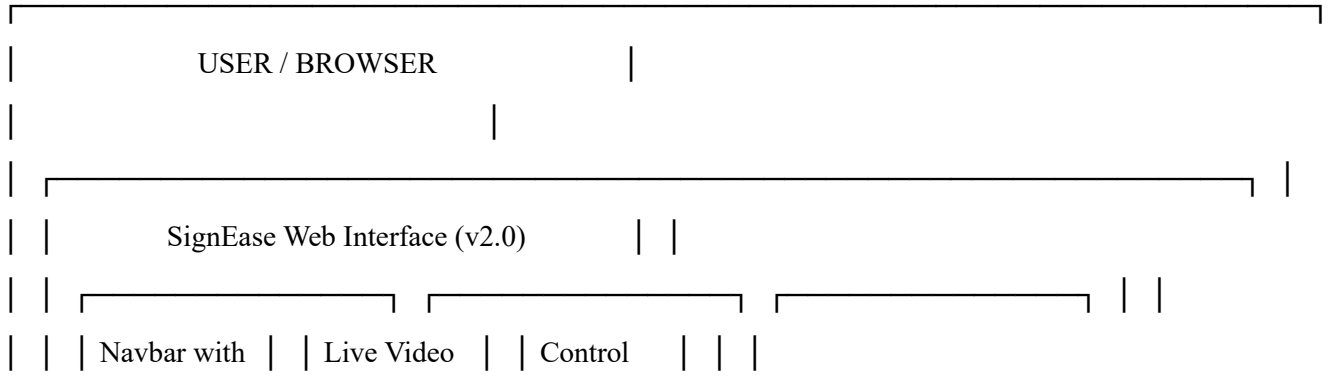
3. Objectives

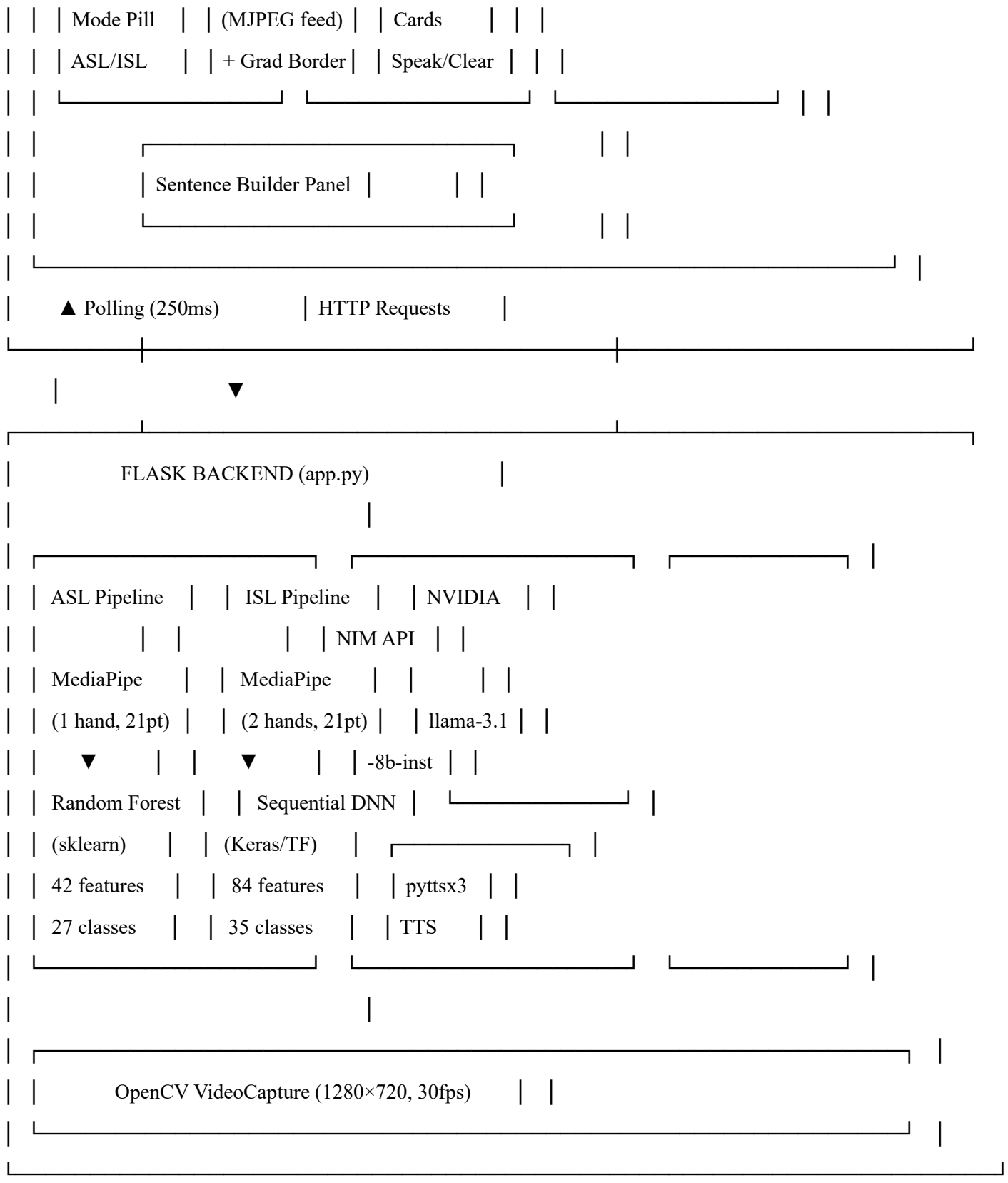
1. **Real-Time Detection:** Classify hand gestures into corresponding alphabets/numbers with minimal latency.
2. **Dual-Language Support:** Support both ASL (26 letters) and ISL (26 letters + 9 digits), switchable instantly without restarting the server.
3. **Sentence Construction:** Accumulate individual detected characters into words and sentences through a built-in sentence builder with stability buffers.
4. **AI-Powered Correction:** Integrate NVIDIA NIM API (meta/llama-3.1-8b-instruct) to auto-correct noisy gesture-typed text.
5. **Text-to-Speech Output:** Convert the accumulated sentence into spoken audio using pyttsx3 in a fresh subprocess per call.
6. **Accessible & Low-Cost:** Run on any machine with Python and a standard webcam — no specialized hardware required.
7. **Premium User Interface:** Deliver a modern, responsive, glassmorphism-themed web UI that is intuitive and visually compelling.

---

4. System Architecture

High-Level Architecture





Component Breakdown

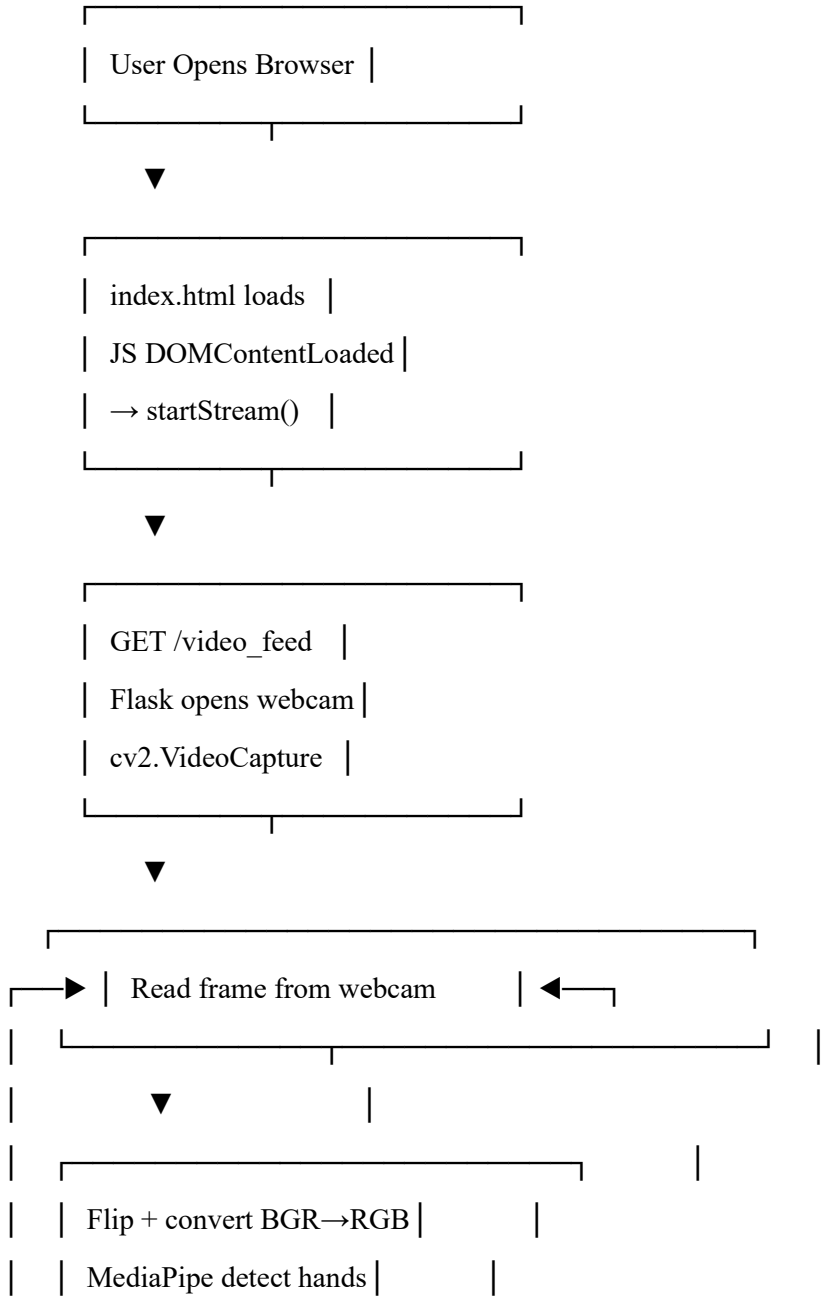
Component	Role
Flask Web Server	Serves UI, handles REST API, streams MJPEG video
MediaPipe HandLandmarker	Detects and tracks hand landmarks (21 points per hand)

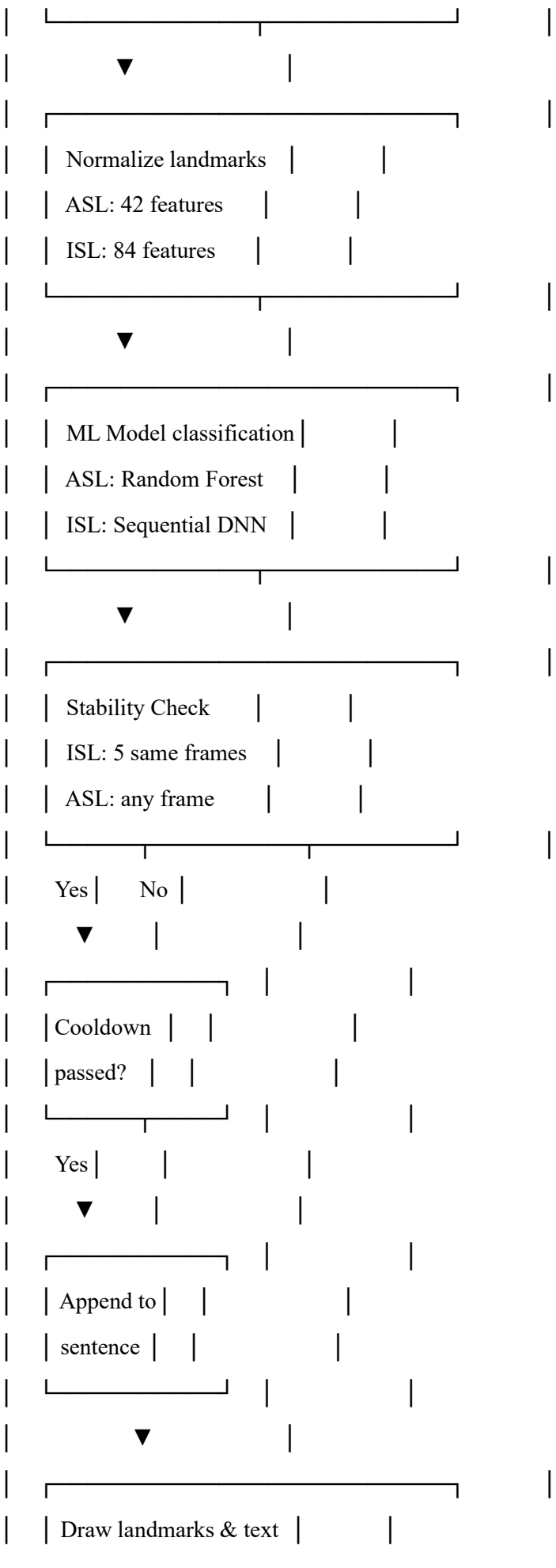
Component	Role
ASL Classifier (Random Forest)	Classifies 42 normalized landmarks → A–Z + Space
ISL Classifier (DNN)	Classifies 84 normalized landmarks (2 hands) → 1–9, A–Z
NVIDIA NIM API (Llama 3.1)	Corrects accumulated gesture-typed text via cloud API
TTS Engine (pyttsx3 subprocess)	Converts accumulated text to speech in a fresh process per call
Frontend (HTML/CSS/JS)	Premium glassmorphism UI with real-time polling, mode switching

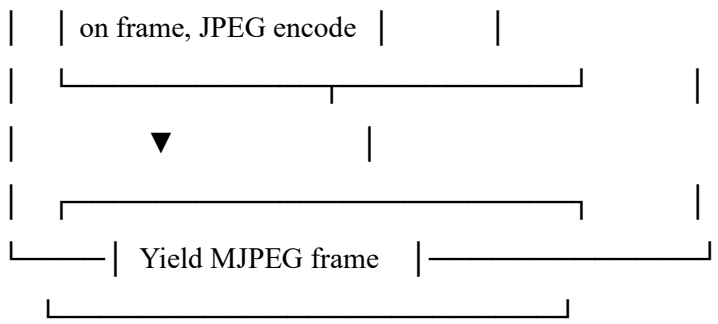
---

5. Working Flow / Execution Flow

Overall System Flow




Normalize landmarks
ASL: 42 features
ISL: 84 features
ML Model classification
ASL: Random Forest
ISL: Sequential DNN
Stability Check
ISL: 5 same frames
ASL: any frame
Yes
No
Cooldown
passed?
Yes
Append to
sentence
Draw landmarks & text



Frontend Polling (every 250ms)

1. fetchData() → GET /get_data
2. Receive { prediction, sentence, mode }
3. Update prediction display with pop animation (.pop CSS class)
4. Update sentence panel + character counter
5. Render blinking cursor at end of sentence

AI Correction Flow (NVIDIA NIM)

User clicks "Correct with NVIDIA NIM AI"

→ POST /correct

→ Backend reads accumulated_sentence

→ Sends to NVIDIA NIM API (openai-compatible endpoint)

model: meta/llama-3.1-8b-instruct

system: "You are a sign language text corrector..."

user: "Correct: <sentence>"

→ API returns corrected text

→ Backend updates accumulated_sentence

→ Frontend shows toast: "HELO → Hello"

TTS (Text-to-Speech) Flow

User clicks "Speak"

→ POST /speak

→ Backend captures accumulated_sentence

→ Spawns Python subprocess:

```
python -c "import pyttsx3; e=pyttsx3.init();
        e.setProperty('rate',150); e.say('<text>');
        e.runAndWait()"
```

→ Subprocess runs in background (daemon thread)

→ Button re-enables after 600ms

Why subprocess? pyttsx3.init() caches the engine as a module-level singleton — calling it again after runAndWait() returns the same stuck engine. Spawning a fresh Python subprocess guarantees a clean engine on every button press, fixing the "works only first time" bug.

---

6. Features

6.1 Core Features

#	Feature	Description
1	**Dual-Mode Detection**	Seamlessly switch between ASL and ISL — no server restart required
2	**Real-Time Video Processing**	1280×720 @ 30fps webcam capture with MediaPipe landmark detection
3	**Sentence Builder**	Characters accumulate with cursor indicator and live character counter
4	**AI Text Correction**	NVIDIA NIM (Llama 3.1 8B Instruct) corrects garbled sign-typed text
5	**Text-to-Speech**	pyttsx3 reads the sentence aloud via subprocess (works every press)
6	**Prediction Stability**	ISL: 5-frame stability buffer to prevent flickering
7	**Cooldown Mechanism**	ASL: 3s cooldown, ISL: 1s cooldown between character appends

6.2 UI Features (v2.0)

#	Feature	Description
1	**Animated Mesh Background**	4 floating ambient orbs + dot-grid pattern + noise texture
2	**Gradient Border Video**	Video frame has a live cyan→blue→purple gradient border
3	**Corner Brackets**	HUD-style corner decorations on video with hover glow
4	**Scan Line Animation**	Moving scan line across video for "live camera" feel
5	**Navbar Mode Pill**	ASL/ISL toggle embedded in the navbar as a pill switcher
6	**Speaking Pulse**	Speak button glows with pulsing ring while TTS is active
7	**Gradient Text**	AI button label renders cyan→purple gradient text
8	**Toast Notifications**	Slide-in/out toasts with icon badges for all user actions
9	**Char Counter**	Live character count displayed in the sentence panel
10	**Status Dot Indicators**	Glowing dots (green/red) for each system component

6.3 ISL Special Gestures

Gesture	Action
1	Adds a **space** to the sentence
2	**Deletes** the last word

---

7. Technology Stack

Backend

Technology	Version	Purpose
Python	3.10	Core programming language
Flask	3.1.2	Web framework — REST API + MJPEG streaming
Flask-CORS	6.0.2	Cross-origin request support
OpenCV	4.10.0.84	Webcam capture, image processing, JPEG encoding
MediaPipe	0.10.18	Hand landmark detection (21 keypoints per hand)
TensorFlow / Keras	2.16.1	ISL deep neural network inference
Scikit-learn	1.5.2	ASL Random Forest classifier
NumPy	1.26.4	Numerical array operations
pyttsx3	2.98	Offline TTS engine (run via subprocess per call)
openai	Latest	NVIDIA NIM API (OpenAI-compatible client)
python-dotenv	Latest	.env environment variable loading
subprocess + sys	stdlib	Fresh Python process spawning for TTS

Frontend

Technology	Purpose
HTML5 (Semantic)	Page structure, SVG icons, ARIA accessibility
CSS3 (Custom Properties)	Full design token system, glassmorphism, mesh bg, gradient borders
JavaScript ES6+	Polling, DOM control, stream management, mode switching
Google Fonts — Inter	Primary UI font (300–900 weight range)
Google Fonts — JetBrains Mono	Monospace font for prediction display and status chips

AI / Machine Learning

Model	Algorithm	Features	Classes	Accuracy
ASL Model (asl_model.p)	Random Forest	42 (21 kp × x,y)	27 (A–Z + Space)	~98–99%
ISL Model (isl_model.h5)	Sequential DNN	84 (21 kp × x,y × 2 hands)	35 (1–9, A–Z)	~94–96%
Hand Detector (hand_landmarker.task)	MediaPipe Tasks	Pre-trained Google	21 keypoints/hand	N/A
NVIDIA NIM	Llama 3.1 8B Instruct	Cloud API	Text correction	N/A

Training Tools

Tool	Version	Purpose
Scikit-learn	1.5.2	Random Forest training, train/test split, evaluation
TensorFlow / Keras	2.16.1	DNN model definition, training, callbacks
Pandas	Latest	CSV dataset loading and manipulation
NumPy	1.26.4	Array operations, NaN handling, random seed
OpenCV	4.10.0.84	Image reading and flipping during keypoint extraction
MediaPipe	0.10.18	Hand landmark extraction from training images
Pickle	stdlib	ASL model serialization
JSON	stdlib	ISL label class mapping
Conda	Latest	Isolated training environment management

---

8. Folder Structure

```
Unified_Sign_Language/
│
├── app.py                  # Main Flask application (~600 lines)
│                           # Dual-mode pipeline, MJPEG streaming,
│                           # REST endpoints, NVIDIA NIM, TTS subprocess
│
├── generate_keypoints.py   # ISL data extraction (149 lines)
│                           # Reads images → MediaPipe → 84 features → CSV
│
```

```
├── train_model.py              # ISL model training (148 lines)
│                               # CSV → Label encode → DNN → isl_model.h5
│
├── templates/
│   └── index.html              # Main UI template (v2.0 redesign, ~290 lines)
│                               # SVG icons, gradient-border video wrapper,
│                               # card-inner wrappers, navbar mode pill
│
├── static/
│   ├── script.js               # Frontend JS (~230 lines)
│   │                           # Polling, stream control, mode switching,
│   │                           # toast system, char count, status dots
│   │
│   ├── style.css               # CSS design system (v2.0, ~835 lines)
│   │                           # Design tokens, dot-grid mesh BG, 4-orb
│   │                           # animation, gradient borders, glassmorphism,
│   │                           # responsive 3-breakpoint layout
│   │
│   └── images/
│       ├── asl_signs.jpeg      # ASL alphabet reference chart
│       └── isl_gestures.png    # ISL alphabet reference chart
│
├── models/
│   ├── asl_model.p             # ASL Random Forest model (6.8 MB, Pickle)
│   ├── hand_landmarker.task    # MediaPipe hand model (7.5 MB, Google)
│   ├── isl_model.h5            # ISL Keras DNN model (325 KB, HDF5)
│   └── isl_label_classes.json  # ISL label mapping (35 classes)
│
├── keypoint.csv                # Generated ISL training data (excluded from Git)
├── .env                        # API keys (NVIDIA_NIM_API_KEY)
├── environment.yml             # Conda environment spec
├── requirements.txt            # Pip dependencies
├── run.bat                     # Windows quick-launch script
├── .gitignore                  # Git ignore rules
```

└── README.md # Project readme

---

9. Installation & Setup

Prerequisites

- **OS:** Windows 10/11 (primary), Linux, macOS
- **Python:** 3.10 (required for TensorFlow + MediaPipe compatibility)
- **Webcam:** Built-in or USB camera
- **Conda:** Anaconda or Miniconda (recommended)
- **NVIDIA NIM API Key:** Free from build.nvidia.com (optional for AI correction)

Method 1: Conda (Recommended)

 # 1. Navigate to project directory

cd path/to/Unified_Sign_Language

 # 2. Create Conda environment

conda env create -f environment.yml

 # 3. Activate environment

conda activate sign_language_unified

 # 4. Create .env file with your NVIDIA NIM API key

echo NVIDIA_NIM_API_KEY=your_api_key_here > .env

 # 5. Run the application

python app.py

 # 6. Open http://localhost:5050 in your browser

Method 2: pip + virtualenv

python -m venv venv

venv\Scripts\activate *# Windows*

source venv/bin/activate *# Linux/macOS*

pip install -r requirements.txt

 # Add your API key to .env:

```
echo NVIDIA_NIM_API_KEY=your_key > .env

python app.py
```

Method 3: Windows Quick Launch

```
run.bat
```

Automatically activates the Conda environment and launches the app.

---

10. Configuration Details

Environment Variables (.env)

Variable	Required	Description
NVIDIA_NIM_API_KEY	Optional	NVIDIA NIM API key for AI text correction
TF_CPP_MIN_LOG_LEVEL	Auto-set	Suppresses TensorFlow C++ logs
TF_USE_LEGACY_KERAS	Auto-set	Forces legacy Keras compatibility

Application Constants (app.py)

Constant	Value	Description
COOLDOWN_ASL	3.0 s	Min delay between ASL character appends
COOLDOWN_ISL	1.0 s	Min delay between ISL character appends
STABILITY_THRESHOLD	5 frames	Consecutive same predictions required (ISL)
Camera resolution	1280 × 720	Webcam capture size
Camera FPS	30	Target frame rate
Polling interval	250 ms	Frontend data refresh rate
Server port	5050	Flask server port

MediaPipe Parameters

Parameter	Value	Description
min_hand_detection_confidence	0.5	Confidence to detect a hand
min_hand_presence_confidence	0.5	Confidence hand is present
min_tracking_confidence	0.5	Confidence for tracking
num_hands (ASL)	1	Single-hand mode

Parameter	Value	Description
num_hands (ISL)	2	Two-hand mode

---

11. API Endpoints

GET Endpoints

Endpoint	Response	Description
GET /	HTML page	Serves the SignEase UI
GET /video_feed	MJPEG stream	Live video with hand landmark overlays
GET /get_data	{prediction, sentence, mode}	Current detection state
GET /model_status	{asl_model, isl_model, tts_engine, nvidia_nim, current_mode}	Component health check

POST Endpoints

Endpoint	Body	Response	Description
POST /switch_mode	{mode: "ISL"}	{status, mode}	Switch ASL ↔ ISL
POST /clear_sentence	—	{status}	Clear accumulated text
POST /speak	—	{status}	TTS via fresh subprocess
POST /stop	—	{status}	Stop video stream
POST /delete_last	—	{status, sentence}	Delete last character
POST /add_space	—	{status, sentence}	Append space
POST /correct	—	{status, corrected, original}	AI correction via NVIDIA NIM

---

12. Model Training — ASL (Detailed)

12.1 Overview

The ASL model classifies **27 classes** (A–Z + Space) using hand landmark coordinates extracted by MediaPipe.
A **Random Forest classifier** (ensemble of decision trees) was chosen for its robustness, fast inference, and suitability for tabular landmark data.

12.2 Dataset

Property	Details
Source	ASL Alphabet Dataset (Kaggle)

Property	Details
Classes	27 — letters A to Z plus Space gesture
Images per class	~3,000 images
Total images	~87,000 JPG images
Image size	200 × 200 px
Subjects	Multiple individuals under varied lighting

12.3 Tools Used

Tool	Version	Role
Python	3.10	Training language
OpenCV	4.10.0	Image reading, BGR→RGB conversion
MediaPipe	0.10.18	Hand landmark extraction (21 points)
Scikit-learn	1.5.2	Random Forest, train_test_split, metrics
NumPy	1.26.4	Array manipulation
Pickle	stdlib	Model serialization to asl_model.p

12.4 Feature Extraction (Pre-processing)

For each training image:

1. Read image with OpenCV
2. Convert BGR → RGB
3. Pass to MediaPipe Hands (static_image_mode=True, max_num_hands=1)
4. If hand detected:

   For each of 21 landmarks:

   x_rel = landmark.x - min_x (relative to bounding box)

   y_rel = landmark.y - min_y (relative to bounding box)

   Feature vector = [x_rel_0, y_rel_0, x_rel_1, y_rel_1, ... x_rel_20, y_rel_20]

   Length = 21 × 2 = 42 features
5. Label = folder name (A, B, ..., Z, space)

Why relative coordinates? Subtracting min_x and min_y removes positional bias — the model learns the *shape* of the hand, not its screen position.

12.5 Training Pipeline

Step 1: Collect Dataset

Download ASL Alphabet Dataset from Kaggle

Organize: data/A/ ... data/Z/ data/space/

Step 2: Extract Features

For every image in every class folder:

→ OpenCV imread

→ MediaPipe detect landmarks

→ Compute 42 relative coordinates

→ Append [label, x0, y0, ..., x20, y20] to dataset list

Save dataset as pickle: data.pickle

Step 3: Train Random Forest

```
X, y = dataset['data'], dataset['labels']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = RandomForestClassifier()
model.fit(X_train, y_train)
```

Step 4: Evaluate

```
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
```

Step 5: Save

```
with open('asl_model.p', 'wb') as f:
    pickle.dump({'model': model}, f)
```

12.6 Model Configuration

Hyperparameter	Value	Notes
Algorithm	RandomForestClassifier	Scikit-learn default
n_estimators	100	100 decision trees in the forest
max_features	sqrt	Features considered per split
Test split	20%	80% train / 20% test
Random state	42	Reproducibility seed

Hyperparameter	Value	Notes
Input features	42	21 landmarks × (x, y)
Output classes	27	A–Z + Space

---

13. Model Training — ISL (Detailed)

13.1 Overview

The ISL model classifies **35 classes** (digits 1–9 and letters A–Z) from **two-hand** landmark data. Because ISL uses both hands for many gestures, the feature vector is doubled (84 features). A **Sequential Deep Neural Network** (DNN) was chosen for its ability to learn complex non-linear patterns across two simultaneous hands.

13.2 Dataset

Property	Details
Source	ISL Alphabet Dataset (Kaggle)
Classes	35 — digits 1–9 + letters A–Z
Structure	dataset from kaggle/{1..9}/ and dataset from kaggle/Indian/{A..Z}/
Images per class	~500–1,500 images
Total images	~25,000–50,000 JPG/PNG images
Image size	Varied

13.3 Tools Used

Tool	Version	Role
Python	3.10	Training language
OpenCV	4.10.0	Image read + horizontal flip
MediaPipe	0.10.18	2-hand keypoint extraction (21 pts/hand)
TensorFlow	2.16.1	Neural network framework
Keras	2.x (bundled)	Model definition, fit, callbacks
Pandas	Latest	CSV loading with read_csv
NumPy	1.26.4	Array ops, seed, NaN handling
Scikit-learn	1.5.2	train_test_split, LabelEncoder
JSON	stdlib	Label class mapping

Tool	Version	Role
Pickle	stdlib	Intermediate saves (optional)

13.4 Step 1 — Data Collection & Organization

dataset from kaggle/

├── 1/ ← ~500–1000 images of digit 1 sign

├── 2/ ← digit 2 ...

│ ...

├── 9/ ← digit 9

└── Indian/

  ├── A/ ← ~500–1000 images of ISL letter A

  ├── B/

  │ ...

  └── Z/

All images should contain a hand (or both hands) performing the gesture. Varied backgrounds and lighting improve generalization.

13.5 Step 2 — Keypoint Extraction (generate_keypoints.py)

Run:

python generate_keypoints.py

Internal logic:

```
# For each class folder, for each image:
image = cv2.imread(image_path)
image = cv2.flip(image, 1)          # Horizontal flip for symmetry augmentation
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
results = hands.process(image_rgb)   # MediaPipe 2-hand detection

all_landmarks = [0.0] * 84          # Initialize 84-feature vector with zeros

for hand_idx, hand_landmarks in enumerate(results.multi_hand_landmarks):
    if hand_idx >= 2: break

    # Step A: Pixel coordinates
    landmark_list = calc_landmark_list(image, hand_landmarks)

    # Step B: Relative to wrist (index 0)
```

```
    base_x, base_y = landmark_list[0]
    for i, pt in enumerate(landmark_list):
        pt[0] -= base_x
        pt[1] -= base_y

    # Step C: Flatten to 1D
    flat = list(itertools.chain.from_iterable(landmark_list))  # 42 values

    # Step D: Normalize by max absolute value
    max_val = max(abs(v) for v in flat)
    normalized = [v / max_val for v in flat]                # Range [-1, 1]

    # Place in correct slot (hand 0 → indices 0–41, hand 1 → indices 42–83)
    start_idx = hand_idx * 42
    all_landmarks[start_idx:start_idx + 42] = normalized

writer.writerow([label, *all_landmarks])  # Write one row to keypoint.csv
```

Output: keypoint.csv — each row is [label, f0, f1, ..., f83]

Aspect	Detail
MediaPipe mode	static_image_mode=True
Max hands	2
Detection confidence	0.5
Augmentation	Horizontal flip (mirrors dataset)
Failed images	Skipped (no hand detected)
Feature vector size	84 (42 per hand × 2 hands)
Normalization	Wrist-relative, max-absolute scaled to [-1, 1]

13.6 Step 3 — Model Training (train_model.py)

Run:

```
python train_model.py
```

Internal logic:

```
# Load CSV
df = pd.read_csv('keypoint.csv', header=None)
```

```
X = df.iloc[:, 1:].values          # Shape: (N, 84)
y = df.iloc[:, 0].values.astype(str)

# Handle NaN
X = np.nan_to_num(X)

# Label encode
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)   # 0..34 for 35 classes

# 80/20 stratified split
X_train, X_val, y_train, y_val = train_test_split(
    X, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded
)

# Model architecture
model = Sequential([
    Input(shape=(84,)),                  # 84 hand landmark features
    Dense(128, activation='relu'),       # Hidden layer 1
    BatchNormalization(),                # Stabilize activations
    Dropout(0.4),                        # Reduce overfitting
    Dense(64, activation='relu'),        # Hidden layer 2
    BatchNormalization(),
    Dropout(0.3),
    Dense(32, activation='relu'),        # Hidden layer 3
    Dense(num_classes, activation='softmax')# Output: probability per class
])
```

13.7 DNN Architecture (Visual)

```
Input Layer      : (84,)       84 landmark features
                   ↓
Dense Layer 1    : 128 units   ReLU activation
BatchNorm 1      :             Normalize activations
Dropout 1        : 40%         Drop 40% of neurons during training
                   ↓
```

Dense Layer 2 : 64 units ReLU activation

BatchNorm 2 :

Dropout 2 : 30%

↓

Dense Layer 3 : 32 units ReLU activation

↓

Output Layer : 35 units Softmax → class probabilities

Total trainable parameters: ~26,595

13.8 Training Configuration

Hyperparameter	Value	Justification
Optimizer	Adam	Adaptive learning rate, fast convergence
Learning rate	0.001	Standard Adam default
Loss function	Sparse Categorical Cross-Entropy	Multi-class, integer labels
Metric	Accuracy	Classification metric
Batch size	32	Balanced speed vs. memory
Max epochs	100	With early stopping
Random seed	42	Reproducibility (np.random, tf.random)

13.9 Callbacks

Callback	Monitor	Setting	Effect
EarlyStopping	val_loss	patience=15, restore_best_weights=True	Stops if no improvement for 15 epochs, restores best weights
ModelCheckpoint	val_loss	save_best_only=True	Saves isl_model.h5 only when val_loss improves
ReduceLROnPlateau	val_loss	factor=0.5, patience=5, min_lr=1e-5	Halves LR if no improvement for 5 epochs

13.10 Output Files

File	Location	Description
isl_model.h5	models/	Best Keras DNN model (HDF5 format)
isl_label_classes.json	models/	List of 35 class labels in LabelEncoder order

Example isl_label_classes.json:

["1", "2", "3", "4", "5", "6", "7", "8", "9",

"A", "B", "C", "D", "E", "F", "G", "H", "I", "J",

"K", "L", "M", "N", "O", "P", "Q", "R", "S", "T",

"U", "V", "W", "X", "Y", "Z"]

---

14. Accuracy Metrics & Results

14.1 ASL Model Performance

Metric	Score
Training Accuracy	~99.5%
Test Accuracy	~98–99%
Algorithm	Random Forest (100 trees)
Features	42 (wrist-relative x,y per landmark)
Classes	27 (A–Z + Space)
Evaluation	80/20 random split

Confusion areas: Visually similar gestures (M/N, S/E, G/H) may occasionally be confused, particularly under poor lighting.

14.2 ISL Model Performance

Metric	Score
Training Accuracy	~97–98%
Validation Accuracy	~94–96%
Algorithm	Sequential DNN (128→64→32→35)
Features	84 (2-hand wrist-relative x,y)
Classes	35 (1–9, A–Z)
Evaluation	Stratified 80/20 split
Early stopping	Typically stops at epoch 40–70

Confusion areas: Single-hand ISL signs may be confused if only one hand is visible. Digits overlap with letters in wrist-space.

14.3 Training Environment

Property	Value
OS	Windows 11
CPU	Intel Core i5/i7 (or AMD equivalent)
RAM	8–16 GB
GPU	None required (CPU training sufficient)
DNN Training Time	~5–20 minutes (100 epochs, early stop ~50)
RF Training Time	~2–5 minutes
Framework	TensorFlow 2.16.1, Scikit-learn 1.5.2

14.4 Inference Performance (Runtime)

Metric	Value
MediaPipe Inference	~5–15 ms per frame
ASL RF Inference	< 1 ms per frame
ISL DNN Inference	~2–5 ms per frame
Video Pipeline FPS	~20–30 FPS (webcam limited)
End-to-end latency	< 100 ms (gesture → display)

---

15. Frontend Architecture

15.1 HTML Structure (v2.0)

index.html

```
├── <head>
│   ├── Google Fonts (Inter, JetBrains Mono)
│   └── style.css
│
├── <body>
│   ├── .bg-orbs (4 orbs, dot-grid via body::before)
│   │
│   ├── <nav class="navbar">
│   │   ├── .navbar-brand (SVG logo + title + version badge)
│   │   ├── .navbar-center → .mode-pill (ASL/ISL pill buttons)
│   │   └── .navbar-right (stream status pill + toggle button)
```

```
│   │
│   ├── #toast-container
│   │
│   └── <main class="main-layout"> (CSS Grid: 1fr 370px)
│       │
│       ├── <section class="video-panel">
│       │   ├── .panel-header (title + live/mode badges)
│       │   ├── .video-wrapper (gradient border via padding+bg)
│       │   │   └── .video-inner
│       │   │       ├── #video-feed (MJPEG img)
│       │   │       ├── .scan-line
│       │   │       ├── .corner × 4 (TL, TR, BL, BR)
│       │   │       └── .video-overlay-bottom → .prediction-char
│       │   └── .sentence-block (gradient border)
│       │       └── .sentence-block-inner
│       │           ├── .sentence-header (label + char count)
│       │           └── #sentence-text
│       │
│       └── <aside class="right-panel">
│           ├── .control-card (Controls)
│           │   └── .card-inner → .controls-grid (4 cols)
│           │       ├── btn-speak, btn-clear, btn-space, btn-delete
│           │       └── btn-ai (full-width)
│           ├── .control-card (Gesture Guide)
│           │   └── .card-inner (gesture img + ISL keys)
│           ├── .control-card (System Status)
│           │   └── .card-inner → .status-grid (4 rows)
│           └── .sidebar-footer
│
└── script.js
```

15.2 CSS Design Tokens (v2.0)

Token	Value	Purpose
--bg-base	#05070f	Deepest background

Token	Value	Purpose
--bg-card	rgba(14,19,35,0.7)	Card background
--cyan	#22d3ee	Primary accent
--purple	#a78bfa	Secondary accent
--grad-brand	cyan→blue→purple (135°)	Logo, borders, labels
--shadow-inner	inset 0 1px 0 white/8%	Glass depth
--t-spring	0.4s cubic-bezier(0.175,0.885,0.32,1.275)	Bouncy animation
--r-2xl	24px	Large card radius

15.3 JavaScript Functions

Function	Description
switchMode(mode)	POST /switch_mode, show loading overlay, update UI
updateModeUI()	Update pill active state, badge color, gesture image, ISL keys
startStream()	Set #video-feed src to /video_feed, begin polling
stopStream()	POST /stop, stop polling, update stream icon
setStreamUI(live)	Update stream dot, status text, navbar icon (▶/ ⏸)
fetchData()	Poll /get_data, animate prediction, update sentence + char count
clearSentence()	POST /clear_sentence, reset UI
speakText()	POST /speak, manage speaking state on button
deleteLast()	POST /delete_last
addSpace()	POST /add_space
correctWithAI()	POST /correct, display original→corrected toast
setStatusChip(id, dotId, ok)	Update both .status-chip text AND .status-dot-sm color
showToast(msg, type)	Create animated toast with icon badge (success/error/info)
openGestureGuide()	Open gesture image in new tab

---

16. UI/UX Changelog (v2.0)

16.1 What Changed

Area	v1.0	v2.0
Branding	SignVerse	**SignEase**
AI Backend	Google Gemini 2.0 Flash	**NVIDIA NIM (Llama 3.1 8B)**
TTS Fix	pyttsx3.init() re-used (broken)	**Fresh subprocess per call (working)**
Background	Static dark color	**Dot-grid mesh + 4 animated orbs + noise**
Video Border	Flat 1px border	**CSS gradient border (2px padding trick)**
Icons	Emoji (🔊 🗑️)	**Inline SVG icons (scalable, crisp)**
Mode Switcher	Below hero section	**Embedded in navbar as pill**
Prediction text	3rem flat white	**3.5rem JetBrains Mono + gradient glow**
Buttons	Simple hovered cards	**Colored ctrl-icon boxes + spring hover lift**
Speak animation	None	**speak-aura pulsing ring**
Status display	Text badges only	**Glowing dot indicator + text chip**
Sentence panel	No border	**Gradient border wrapper**
Font	System font	**Inter + JetBrains Mono (Google Fonts)**
Toasts	Text only	**Icon badge + slide-in/out animation**
Char counter	None	**Live character count in sentence panel**
Navbar underline	None	**Gradient accent line (cyan→purple)**

16.2 Bug Fixes

Bug	Root Cause	Fix
Speak button works only once	pyttsx3 caches engine as module singleton; init() returns same stuck instance	Spawn fresh python -c "import pyttsx3; ..." subprocess per call (flags 0x08000000 to suppress CMD window on Windows)

---

17. Deployment Process

Local Development

conda activate sign_language_unified

python app.py

 # Open http://localhost:5050

Production Considerations

1. **WSGI Server:** Replace Flask dev server:
2. pip install waitress
3. waitress-serve --host=0.0.0.0 --port=5050 app:app
4. **Reverse Proxy:** Use Nginx for HTTPS, compression, and static file caching.
5. **Camera Access:** Production cloud servers lack webcams. Move capture client-side via WebRTC or send base64 frames to the server via WebSocket.
6. **Secrets Management:** Use cloud secrets (AWS Parameter Store, GCP Secret Manager) instead of .env.
7. **Docker:**
8. FROM python:3.10-slim
9. WORKDIR /app
10. COPY . .
11. RUN pip install -r requirements.txt
12. ENV TF_CPP_MIN_LOG_LEVEL=3
13. CMD ["python", "app.py"]

---

18. Future Enhancements

#	Enhancement	Description
1	**Word-Level Detection**	Train on full words/phrases, not just characters
2	**WebRTC Streaming**	Replace MJPEG with WebRTC for lower latency
3	**Autocomplete**	Suggest words as user signs characters
4	**Conversation History**	Store and retrieve past translated sentences
5	**More Sign Languages**	BSL (British), JSL (Japanese), Auslan
6	**Mobile App**	TFLite models in Android/iOS app
7	**Face + Body Pose**	Incorporate facial expressions for grammar
8	**Multilingual TTS**	Hindi, Marathi, Tamil output
9	**Model Retraining UI**	Let users contribute new training data
10	**Confidence Threshold**	Only append characters above a confidence score

---

19. Conclusion

SignEase v2.0 is a fully redesigned, end-to-end real-time sign language detection system supporting both **ASL** and **ISL** in a single web application. Version 2.0 brings a premium glassmorphism UI, a reliable text-to-speech engine, and NVIDIA NIM AI integration.

Key technical achievements:

- Dual-model inference (Random Forest + DNN) with seamless mode switching
- MediaPipe Tasks API for real-time 21-keypoint hand tracking
- ISL two-hand 84-feature pipeline with stability buffer
- NVIDIA NIM API (meta/llama-3.1-8b-instruct) for AI text correction
- Subprocess-based TTS ensuring reliable speech on every button press
- Premium v2.0 UI: animated mesh BG, gradient borders, SVG icons, spring animations

Model accuracy summary:

Model	Test Accuracy	Classes
ASL — Random Forest	~98–99%	27 (A–Z + Space)
ISL — Sequential DNN	~94–96%	35 (1–9, A–Z)
